What is the difference between MLOps, ModelOps & AIOps

Are you looking to become a MLOps Engineer? Go through 360DigiTMG's MLOps Course in Bangalore.

MLOps is an application of DevOps in building end-to-end Machine Learning algorithms including - Data Collection, Data Pre-processing, Model Building, Model Deployment in Production, Monitoring Model in Production, and Model Periodic Upgradation.

ModelOps is the application of DevOps in handling end to end implementation of any algorithms such as Rule-Based Models. This is a more generic term used

AIOps is building AI applications end to end using DevOps concepts

Define MLOps and how is it different from Data Science?

MLOps is a profession where the entire lifecycle including the deployment and monitoring in production is performed seamlessly. This also means that the Data Science workforce with MLOps skills will be more preferred and this will be the way forward for scaling up the career ladder & earn lucrative salaries that are much higher than typical Data Scientists.

What is the difference between MLOps and DevOps?

MLOps & DevOps have a lot of things in common. However, DevOps include developing and deploying the software application code in production and this code is usually static and does not change rapidly.

MLOps on the other side also includes developing and deploying the ML code in production. However, here the data changes rapidly and the up-gradation of models has to happen more frequently than typical software application code.

What is the difference between MLOps and DataOps?

DataOps is a term coined by IBM with focus on data quality. Sudden change in data will trigger an alarm to the stakeholders for action.

yourself a promising career in MLOps Course in Chennai by enrolling in the MLOps Training and Placement Program offered by 360DigiTMG.

MLOps has DataOps as one of the components and in addition to that it has end to end model development, deployment, monitoring in place.

What are the risks associated with Data Science & how MLOps can overcome the same?

Data Science typically has the following issues:

Model goes down without an alert and becomes unavailable

Model gives incorrect predictions for a given observation that cannot be scrutinized further

Model accuracy decreases further as and how time progresses

Model maintenance also should be done by data scientists, who are expensive

Model scaling across the organization is not easy

These risks can be addressed by using MLOps.

Is model deployment end of ML lifecycle?

Model deployment in production is in the current world being treated as the start of the actual ML lifecycle. Monitoring how the model is performing for a longer duration, how the data is increasing, and how to scale the model for wider organization use is something that is done post deployment. These are the activities which are at the core of ML lifecycle and it at the heart of MLOps.

What is MLOps?

MLOps, a.k.a Machine Learning Operations is an emerging domain within the larger AI/DS/ML space that addresses the problem of operationalizing the ML models. MLOps can be thought of as a practice and culture within software engineering which fundamentally attempts to unify the machine learning/data science model development (Dev) and its subsequent operationalization (Ops). MLOps has some analogies to traditional DevOps but it is also significantly different from that. While DevOps predominantly focuses on operationalizing code and software releases which may not be stateful, MLOps has another complexity added to it - data. That is why MLOps is often referred to as the union of ML + Data + Ops (machine learning, data engineering, and DevOps).

What are the benefits of MLOps?

MLOps has several benefits. Some of them are listed below (in no particular order)

Improves Efficiency - By way of implementing MLOps principles allows both Data Engineers and Data Scientists to have unfettered access to curated and cultivated datasets and exponentially increases their ability to develop models faster

Rinse/Repeat - Because MLOps helps in automating all or most of the tasks/steps in the MDLC (model development lifecycle), data scientists and MLOps engineers can reproduce experiments quickly ensuring models are trained and evaluated properly. Also enables versioning both for models and data.

Improves reliability - Because MLOps practices borrow heavily from DevOps it also ingrains within itself several of CI/CD principles thereby improving code quality and reliability.

Leaves breadcrumbs (audit trail) - The ability to have models and datasets versioned will improve the model audit trail considerably allowing data scientists to fall back on the model which performed better if the newer iteration does not meet expectations

How do you create infrastructure for MLOps?

There are many different ways in which MLOps infrastructure can be created. The core responsibility typically lies outside of the scope of an MLOps engineer. However, for a given set of existing environments, the MLOps engineer can definitely create a tech stack that can be best suited for hosting a successful machine learning platform. For example, if the enterprise has a predominantly AWS-based infrastructure, then it becomes easy to implement MLOps pipelines utilizing AWS Sagemaker framework in conjunction with services like Sagemaker pipelines, Cloudformation, Lambdas for orchestration and Infrastructure as Code. If the enterprise is open, then the best platform for most modern software development firms is leaning towards a Kubernetes (k8s) powered infrastructure. This also enables the ML engineer to adopt Kubeflow which is quickly becoming the de facto MLOps framework of choice for

many ML practitioners. However, creating an infrastructure exclusively for ML models is generally not within the scope of an ML Engineer.

How to create CI/CD pipelines for machine learning?

CI stands for continuous integration and CD stands for continuous deployment. The fundamental feature of having a CI/CD pipeline is to ensure that data scientists and software engineering teams are able to create and deploy error-free code as quickly as possible.

Specifically, a CI/CD pipeline aims to automate and streamline the software deployment process which includes - building code, running tests and deploying new versions of model/application when there are updates/revisions.

CI/CD for machine learning has an added complexity in terms of including data in addition to code. But, it could be achieved through a variety of tools depending on the technical stack the enterprise is using.

If the technical stack is primarily AWS driven, Sagemaker pipelines can stand in for CI/CD pipelines.

Other approaches could be to use Kubeflow pipelines and traditional tools like Jenkins or even Github actions to build CI/CD pipelines.

Also, check this MLOps Course in Pune to start a career in Best MLOps Training Institute.

Explain about model/concept drift.

Model drift, sometimes called concept drift, occurs when the model performance during the inference phase (using real-world data) degrades when compared to its performance during the training phase (using historical, labeled data). It is also known as train/serve skew as the performance of the model is skewed when compared with the training and serving phases. This could be due to many reasons like

The underlying distribution of data has changed

Unforeseen events - like a model trained on pre-covid data is expected to perform much worse on data during the COVID-19 pandemic

Training happened on a limited number of categories but a recent environmental change happened which added another category

In NLP problems the real world data has significantly more number of tokens that are different from training data

To detect model drift, it is always necessary to keep continuously monitoring the performance of the model. If there is a sustained degradation of model performance, the cause needs to be investigated and treatment methods need to be applied accordingly which almost always involves model retraining.

What is MLOps?

MLOps is the intersection of Machine Learning and DevOps principles. There is a growing need in the Data Science and Artificial Intelligence industry to quickly and efficiently deploy ML models into production. With the power of MLOps, Data Scientists can iterate over models fast. Once the best one is evaluated and identified, it can be easily deployed typically on the cloud as containers. On top of that,

MLOps frameworks allow data scientists to track and version their experiments, perform A/B testing, monitor their performance, and log results creating a feedback loop. It is a powerful framework for ML practitioners that can be achieved using Kubeflow, MLFlow, Apache Airflow, and Tensorflow Extended among others.

Difference between DevOps and MLOps?

MLOps has a lot of similarities with DevOps, in that it has origins in the latter. Essentially, MLOps exists because of the inherent differences between software engineering and machine learning projects. DevOps principles for software engineering are fairly robust and well established. But ML projects have some unique features such as:

Exploratory in nature, sometimes there may not be a result or the result is not satisfactory.

Data scientists and ML engineers are researchers and mathematicians who may not have the skills to produce production-quality code.

The added complexity in testing - need to test data validation, the model quality validation, and model validation.

Continuous Integration (CI) is not just about code and components; it also needs to account for models, input data, and its schema.

To learn more about Best MLOps Courses, the best place is 360DigiTMG, with multiple awards in its name 360DigiTMG is the Best place to start your MLOps Course Training in Hyderabad. Enroll now!

Continuous Delivery (CD) is not just a single service or software but an entire ML pipeline (for various stages of the MLDC) which should serve the inference pipeline.

Continuous Training (CT) is unique to MLOps, where the framework has mechanisms in place for retraining and calibrating models periodically.

What is TFDV and how does it help with some of the pertinent challenges of MLOps?

TFX is the open-source version of the data science and initial phases of the MLOps solution developed by Google. It stands for Tensorflow Extended. The main idea behind TFX is ‘standardization’. There are multiple ways to develop a data science solution. But, that leads to issues such as reproducibility, production-friendly, and no way to monitor models post-production deployment.

TFX takes each step of the ML model life cycle and develops components to standardize the code development process. In practice, the ML Model lifecycle has a lot of stages before we begin training the model such as Data ingestion, validation, and transformation commonly bundled in the CRISP-ML stages known as Data Understanding and Data Preparation. TFX emphasizes the importance of validating datasets and asserting the schema, calculating the statistics and distribution of the features, etc. This is done using something called TensorFlow data validation a.k.a TFDV which is both a component of TFX. But, could also be used as a standalone library. Not only does this feature give us the ability to know about the distribution and schema of our data. But, also gives us the ability to compare two datasets that can be used to determine if our train/eval splits are having similar characteristics, etc.

By way of extension, this feature also allows us to compare the training data and the inference data and look for something called ‘data drift’. Data Drift is a condition where the inference data on which predictions are expected do not follow the same distribution as the training data.

Define train/serve skew and some potential ways to avoid them

Most often than not, data is not passed to the modeling phase in its raw format. It needs to be preprocessed and hence undergoes several transformations. Moreover, a lot of machine learning algorithms accept only numerical inputs and aren’t equipped to deal with missing values and outliers. In other cases, they need to be completely transformed into something else like removing/filling missing values, handling outliers, scaling numerical values, encoding categorical features, etc.

The challenge is that all the processing steps need to be repeated when trying to derive inferences because the model expects the data on which predictions need to be issued to be in the same format as the training data.

If the prediction data differs significantly from the training data then it can be argued that there is a train/serve skew.

There are multiple ways to avoid train serve skew like:

Maintain separate module files for data preprocessing (a separate class or module.py file)

Compose a preprocessing graph using TFX transform graph etc

In addition to CI and CD are there any other considerations unique to MLOps?

The introduction of data and mathematical logic (algorithms/models) that are applied to that data makes MLOps an interesting endeavor. Ideally for mos software engineering projects CI/CD should be enough, but MLOps also introduces the following concepts:

Continuous Training: when to re-train your model, how often do you do it, etc.

Continuous Monitoring: is the model performing well in the field.

What can be some of the deployment strategies borrowed from DevOps that can be utilized in MLOPs and how to achieve them?

There are a few concepts from web service deployments that map nicely into strategies for deploying models into production environments, like creating several instances of a live inferencing application for scalability and progressively switching from an older to a newer model. A couple of them are

Blue-Green Deployment

Canary deployment

In a Blue-Green deployment, the newer version of the model is brought into the staging environment that is almost identical to the production environment. In some cases, the environment is the same as the production environment but the traffic is routed differently. If we utilize Kubernetes, it is possible to have a single k8s cluster to route the traffic to a separate (new k8s cluster) - the ‘blue’ deployment while the production traffic is going to older - ‘green’ deployment. This is to allow further testing of the newer

model in a production environment before complete adoption. Once enough confidence is established in the newer model the older version is then moved to ‘green’ status and the process will repeat with any further improvements.

Canary deployment is a bit more involved and usually a lot riskier but it is gaining popularity among the DevOps community. It follows a similar deployment model as the blue-green discussed above but provides the ability to progressively change configuration based on constraints depending on the level of confidence in the newer model. In this case, traffic is routed progressively to the newer model at the same time the previous model is serving predictions. So the two versions are live and processing requests simultaneously, but doing them in different ratios. The reason for this percentage-based rollout is that you can enable metrics and other checks to capture problems in real-time, allowing you to roll back immediately if conditions are unfavorable.

Both of these strategies can be applied by Kubeflow as it natively relies on the Kubernetes environment.

What is MLOps

MLOps is a set of practices whose goal is to deploy and maintain ML models in production reliably and efficiently. MLOps rose as machine learning models became increasingly incorporated into software, similarly to how DevOps practices rose for typical software development.

Tasks in the Production ML Lifecycle

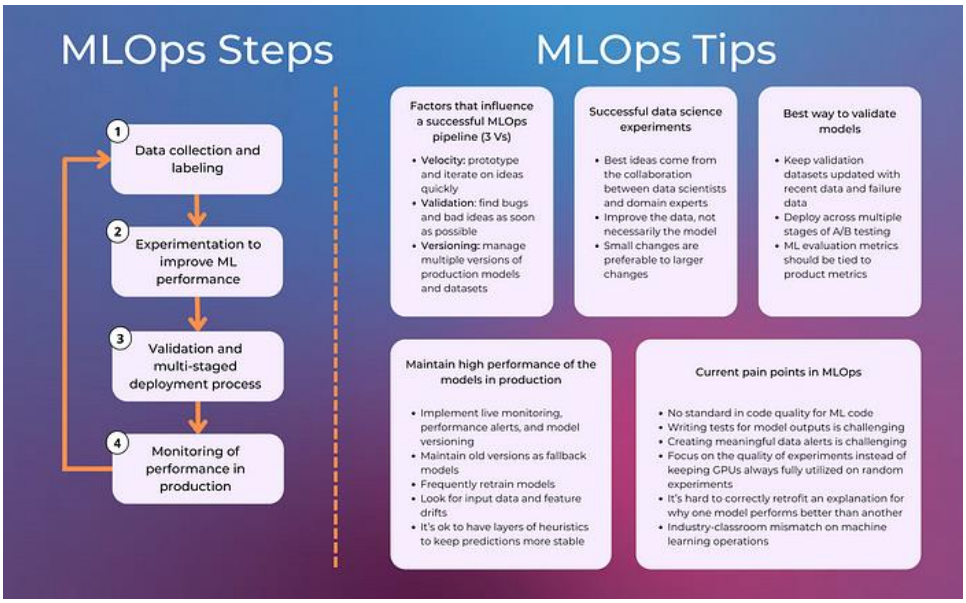
The process of operationalizing ML consists, for the majority of respondents, of a continual loop of the following four steps:

Data collection and labeling: Collection of data spanning multiple data sources, wrangling them into a centralized repository, and data cleaning.

Experimentation to improve ML performance: Make new experiments by creating new features or changing the model architecture.

Validation and multi-staged deployment process: Typically, a trained model is then evaluated on a test set, producing some metrics. These metrics are then compared to those of other models previously trained, and the best model is chosen to be deployed into production. The deployment may be composed of multiple steps, staging the change to increasing percentages of the population (i.e. A/B testing).

Monitoring of performance in production: The deployed model is monitored via live metrics, slicing, and dicing sub-populations to investigate prediction quality. If the model doesn’t perform as expected, it’s substituted by the model previously in production (i.e. a rollback).



Based on these four steps, the respondents answered questions addressing the following issues:

What are the factors that influence a successful MLOps pipeline.

How to do new successful data science experiments.

What is the best way to validate models.

How to maintain high performance of the models in production.

Current pain points in MLOps.

Let's go into detail about each of these.

What are the factors that influence a successful MLOps pipeline

From the interviews it emerged that the success of ML deployments depends mainly on three factors:

Velocity: Since ML is so experimental in nature, it's important to be able to prototype and iterate on ideas quickly (e.g. go from a new idea to a trained model in a day). Data science productivity is often the result of development environments that prioritized high experimentation velocity while allowing to test hypotheses quickly.

Validation: Since errors become more expensive to handle when users see them, it's good to test changes, prune bad ideas, and proactively monitor pipelines for bugs as early as possible.

Versioning: It's helpful to manage multiple versions of production models and datasets for querying, debugging, and minimizing production pipeline downtime. ML engineers respond to buggy models in production by switching the model to a simpler, historical, or retrained version.

An MLOps pipeline that meets the requirements of velocity, validation, and versioning is more likely to have success with ML deployments.

How to do new successful data science experiments

These are the aspects most taken into consideration in creating new experiments:

Best ideas come from the collaboration between data scientists and domain experts: Project ideas (e.g. what new features to use) often come from domain experts, data scientists, and analysts who had already performed a lot of exploratory data analysis.

Improve the data, not necessarily the model: It is often recommended to experiment with variations on the data (e.g. new features) instead of trying different models. Indeed, Andrew Ng suggests companies move from a model-centric approach to a data-centric approach.

Small changes are preferable to larger changes: Similar to traditional code development, it is preferable to make a few changes at a time for faster code reviews, easier validation, and fewer merge conflicts.

What is the best way to validate models

Here are the tips on model validation:

Keep validation datasets updated with recent data and failure data: Many engineers reported processes to analyze live failure modes and update the validation datasets to prevent similar failures from happening again.

Deploy across multiple stages of A/B testing: Several organizations, particularly those with many customers, had a multi-stage deployment process for new models or model changes, progressively evaluating at each stage. The goal is to catch problems as soon as possible before they meet customers.

ML evaluation metrics should be tied to product metrics: Multiple participants stressed the importance of evaluating metrics critical to the product, such as click-through rate or user churn rate, rather than ML-specific metrics alone like MAP.

How to maintain high performance of the models in production

These are the tips for maintaining good production performance:

Implement live monitoring, performance alerts, and model versioning: It's useful to check the performance in production, implement alerts and fallback to a previous model if the new model does not perform as expected.

Maintain old versions as fallback models: Another way to minimize downtime when a model is known to be broken is to have a fallback model to revert to, either an old or a simpler version.

Frequently retrain models: If it is not too expensive to retrain a model, it makes sense to do it daily/weekly to quickly solve data drift problems.

Look for input data and feature drifts: A change in the distribution of the input data is another indicator of a possible decline in model performance.

It's ok to have layers of heuristics to keep predictions more stable: It's common to augment models with a final, rule-based layer to keep live predictions more stable.

Current pain points in MLOps

Here are some common pain points and anti-patterns (i.e. common responses to recurring problems that are usually ineffective and risk being counterproductive) in MLOps:

No standard in code quality for ML code: Since it is important to experiment quickly, bad and undocumented code is often written. In addition, some use Jupyter Notebooks as an environment to experiment and a common code editor to write production code, while others use one or the other for both phases.

Writing tests for model outputs is challenging: In standard software engineering, one can write test cases to cover the space of potential bugs. But for ML, it's hard to categorize bad predictions effectively and write tests for them, because every prediction feels unique.

Creating meaningful data alerts is challenging: MLEs often implement rules that trigger too many alarms, resulting in a lot of work for the people who analyze them. It is not easy to create rules that generate few significant alarms.

Focus on the quality of experiments instead of keeping GPUs always fully utilized on random experiments: Many MLEs try to run as many experiments as possible given their computational resources, ensuring all GPUs are utilized at any given time. While doing as many experiments as possible during the day may seem like good practice, it actually detracts from the real goal of doing more good experiments during the day.

It's hard to correctly retrofit an explanation for why one model performs better than another: One often tries to give a reason why one model performs better than another by intuition, without performing ablation tests. This can lead to incorrect explanations, confusing correlation with causation.

Industry-classroom mismatch on machine learning operations: Many topics about MLOps are only addressed on the job and are not covered in education.

Conclusion and next steps

MLOps is in its infancy and it will take years to arrive at standard practices recognized by the majority of practitioners, as has happened previously for DevOps. Still, the paper "Operationalizing Machine Learning: An Interview Study" did a great job in summarizing what companies are doing nowadays when operationalizing machine learning models.

How to answer the MLOps pipeline interview question

Andrejs

.

Follow

6 min read

.

Sep 3, 2023

If you apply for data science and ML engineering jobs, then you also need to be prepared for answering “How would you design an MLOps pipeline\system…” followed by some specific use-case application. “Structural questions have to be answered in a structured way” — I thought and drafted this plan.

Define system requirements

Usually interviewers provide no details by default, waiting to see how I handle that. In real-life this is a crucial part of a project, which can’t be redone later, but during the interview I see it as a generic part. I try to do it in semi-automatic way and use this time to calm down and think ahead of the next steps specific to the problem at hand.

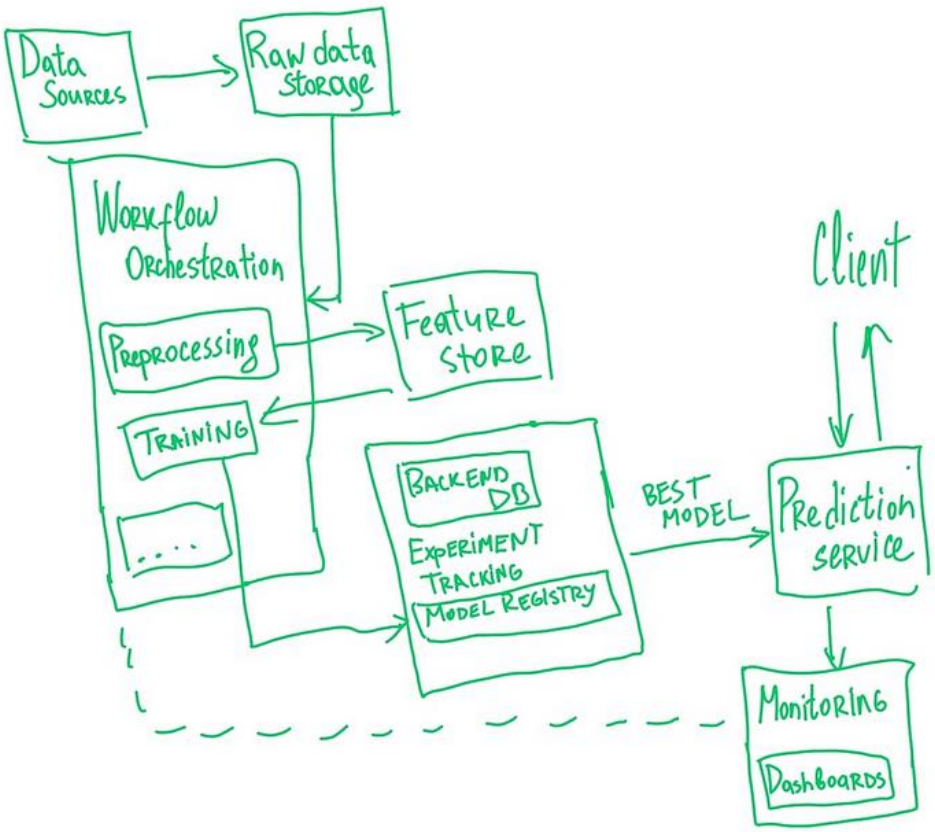
First thing, I clarify (non-)functional requirements. This is a well-determined step, so, everyone, just do your homework!

I ask questions, but do not turn the interview table. I stay in the lead, think out loud and ask rhetoric questions inviting them to comment if needed. Otherwise, I assume (and explicitly mention every assumption) things the way it benefits me — for instance, by not making it too complicated for myself later.
Choose a reasonable number of question to ask depending whether you want to show off as a technology-, business- or product-minded person.
Here are some ideas, but just ask ChatGPT yourself.

— How often, in which ways and formats new data is coming in? What is data retention policy regarding raw data, features, etc.?
— Does the model(s) need to be (re)trained? In which circumstances? Is there anything that can set limitations on model’s architecture?
— What scalability and availability requirements are there? What if the service access is broken?
— Which KPIs need to be monitored? Which actions are taken when data drift is detected?
— How the service is being invoked, batch or online? Who is the client side and which functionality does it have?

If interviewers decided to step in and share specifics information, I make a mental note and try to ask a clarifying question.

I keep timing in mind. This is an important step, but only a pre-requisite, that is, ideally, I need to clarify the anchor-points to get back to in the next steps. It shouldn’t take more than 3–5 minutes if it is just a part of a technical interview or a bit more if it is a longer “system design” interview.



High-level design sketch

Before starting, I let people know that this is a high-level overview and details will follow. I try to keep interviewers informed about my next 1 or 2 steps in the explanation. Hopefully, it creates a perception that I am in control.

I start explaining high level components. For an MLOps pipeline these are: Data management, Model and Experiments management, Deployment management, Monitoring management. I draw some squares on the whiteboard, but leave ample space between to add more details later.
How “high” is too high-level? Well, probably, this is too skinny:

Of course the layout may differ. For instance, if it is a streaming application, this should be reflected already in this draft. The most important is to ensure that this design does not contradict any of the requirements set above. Tell a few sentences about each component functionality and how they are integrated together.

We have an orchestrator which periodically performs batch injection of fresh data from the source by API.
The raw data is assumed to be structured and we store it in the date lake.
It passes through a pre-processing pipeline and feature generation.
If, under certain conditions, we need to retrain a model, experiments and artifacts are logged separately.

The best model from the model registry is used inside the prediction service deployed using managed cloud infrastructure, it is accessible through the RESTful API.
When a client requests a prediction, inputs are accumulated in another database and monitoring service processes the fresh data point, checks for data drift and updates internal dashboards.

Hopefully, I am not interrupted too much yet, otherwise I did smth wrong either in the design or in communication. If so, the “fog of war” should have been dissipated a bit and I have made up my mind on each component’s implementation.

Detailed architecture choices and functionality

Below are some lower-level building blocks, which I would consider. After writing that part, I noticed, that almost any challenge can be solved by a “managed cloud solution”. Shouldn’t be a surprise that any possible ML system (except maybe HFT, where nano-seconds separate winners and losers) can be implemented fully on Azure, AWS or GCP, otherwise, they wouldn’t be vendors of major tech companies in the world.
Though, I need to be ready to explain the choices and make sure the whole pipeline passes integration tests as some combinations would not make sense or be redundant.

Data

Data ingestion — fetch from external APIs, scheduled jobs (Cron, Airflow, Prefect), event trigger (webhooks, IoT, database update)
Storage — cloud-based data lake solutions, on-premises HDFS system, distributed databases like Apache Cassandra.
Versioning, archiving — managed data lake can take care of that, or tools like DVC (but then be aware of their limitations)
Security and access control, encryption — remember, I said in the beginning to be in the lead of requirements and not to put yourself in a difficult position? :)
Preprocessing and feature store — custom Python code, Spark or, again, cloud-based (Google Data Factory)

Models and Experiments

Model Selection — choice of the model because of limitations, prior knowledge or by training many of them (e.g. AutoML); data science essentials are helpful here
Training and finetuning — does the dataset require distributed computing, how data is being fetched and split for training, which tools are used (TF, PyTorch, Optuna)
Automated retraining — when does it happen (scheduled, triggered by CI/CD,)
Model registry — many free solutions with paid perks (MLflow, W&B, Kubeflow)
Experiment tracking — separate from the model registry, essentially a database like Postgres with a UI from one of the solutions above

Deployment

By choosing the deployment style consider:

costs and how expensive is a failed inference task

number of models or their versions deployed simultaneously

whether the inference service requires to have memory about the state

if inference can be delayed and done in batches or has latency requirements, security or regulatory constraints

if it requires automatic scaling

Most popular answer in production nowadays probably is to allow Kubernetes orchestrate the containers. Alternatives are Docker containers, serverless solutions like AWS Lambda and GCP Cloud Run, local servers. More niche solutions are edge devices and other hardware.
In most cases actual invocation happens by means of HTTP requests via an exposed API (FastAPI, Flask are here to help). Talk through the list of endpoints the client side should have access to and what happens in case any of them is called. Explain how deployment will know which model artifacts to deploy and when does the model serving happens

Monitoring

Reiterate again monitoring objectives — regarding model performance, data quality or anything else important like inference time, F1-score or user-feedback rate.
Continue to the tooling, most probably you need something for monitoring and for visualization: Not having very diverse experience, I choose for Prometheus+Evidently+Grafana. As before, there are cloud-native solutions.
Explain what happens when anomaly event is detected and what is your incident response plan — retrain the model, provide baseline output, issue alerts, etc.

Conclusion

In conclusion, I refer back to which requirements the pipeline covers and which need extra attention. If there are questions left, I am ready to improvize now.

https://www.tryexponent.com/blog/machine-learning-system-design-interview-guide

https://towardsdatascience.com/how-to-answer-any-machine-learning-system-design-interview-question-a98656bb7ff0

Question 1: How do you design a machine learning system for a problem with very limited labeled data?

Employ transfer learning, using pre-trained models or fine-tuning on the limited data.

Implement data augmentation techniques to artificially increase the size of the training dataset.

Use semi-supervised learning methods that leverage both labeled and unlabeled data.

Apply active learning, where the model iteratively selects the most informative instances for labeling.

Consider using synthetic data generation techniques to create additional training samples.

2. How do you handle concept drift in a deployed ML model?

Monitor model performance and track drift metrics

Perform regular model retraining with new data

Implement online learning algorithms, if applicable

Utilize ensemble models or model stacking for better adaptability

Alert system to notify engineers when performance degrades

3. How do you ensure data privacy and security when designing a machine learning system?

Use data anonymization techniques (e.g., k-anonymity, l-diversity)

Implement data encryption at rest and in transit